THE UNIVERSITY OF DAR ES SALAAM

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

COURSE: IS 365 - PRACTICAL ASSIGNMENT (LOCAL LLM APPLICATION)

UDSM STUDENT SUPPORT ASSISTANT

TECHNICAL REPORT

LOCAL LLM PIPELINE - FASTAPI, STREAMLIT AND OLLAMA

STUDENT'S DETAILS:

NAME	REGISTRATION NUMBER	PROGRAMME
Charles J Tungaraza	2023-04-13490	BSc Computer Science
Stella Thomas Kahungo	2023-04-03737	BSc Computer Science
Godbless Kaaya	2023-04-03579	BSc Computer Science
Baraka Jimmy Maengesho	2023-04-06473	BSc Computer Science
Frank Elikana Wallace	2023-04-13642	BSc Computer Science
Kelvin George Msuya	2023-04-08929	BSc BIT

1. Introduction

This report describes our IS 365 group project: a small chatbot that helps UDSM students ask questions about campus services. We built it on one laptop using Python, FastAPI, Streamlit, and a local language model through Ollama.

The assistant covers topics like course registration (ARIS), exams, library, ICT, hostel, fees (GePG), academic calendar, and student conduct. We did not try to build the smartest chatbot possible. The main goal was to show the full path from setup → local model → API → simple web interface → testing and logging, which is what a real workplace project would also need.

1.1 Objectives

Our group set five practical objectives for this assignment:

1. Run a language model **locally** on student hardware (no paid cloud API).
2. Expose the model through a **proper REST API** with validation and documentation.
3. Build a **simple web chat** that a non-technical student could use.
4. Improve answer quality using a **UDSM FAQ file** and a **custom system prompt**.
5. Show that the system is **testable and observable** through pytest, logs, and screenshots.

1.2 Scope and limitations

In scope: One-machine prototype, demo FAQ content, keyword-based retrieval, ten API tests, error messages, and logging.

Out of scope: Real ARIS integration, official UDSM policy documents, user login, HTTPS deployment, GPU servers, and fine-tuning the model on university data.

We label all FAQ answers as **demonstration content** unless UDSM staff verify them for production use.

2. Problem statement and solution

At UDSM, students often look for answers in different places: the website, notice boards, ICT office, library desk, or Dean of Students. That takes time, especially for new students who do not yet know which office handles which issue.

Problem: Information is scattered and students repeat the same questions every semester (registration, fees, hostel, exams).

Our solution: A chat page where the student types a question and gets an answer on the same screen. The answer is written by a **local** model (llama3.2:1b), but we guide it using:

1. A **UDSM FAQ file** we wrote (data/university_faq.md)
2. **Keyword search** on that file before each answer (simple RAG)
3. An **improved system prompt** with UDSM rules

2.1 Example questions we tested

Question	Expected topic	Result
How do I register for courses at UDSM?	ARIS registration steps	Answer lists portal login and registration window
How do I pay fees through GePG?	Fee payment	Answer mentions control number and payment channels
What are UDSM library hours?	Library services	Answer gives demo opening hours from FAQ
How do I apply for hostel?	Accommodation	Answer lists application steps from FAQ

The FAQ is **demo content** for this assignment, not official university policy.

2.2 Why a local LLM pipeline?

We chose a self-hosted pipeline because IS 365 focuses on understanding how industry systems are built end-to-end. A student who only uses ChatGPT in the browser does not see the API layer, configuration, logging, or deployment risks. Our project makes those parts visible and testable.

3. Tools and technology choices

We developed on **Windows 10** on a **Dell Inspiron 3543 (8 GB RAM, Intel i7-5500U)**. That limited our choices.

Tool	Role	Why we used it
Python 3.10+	Main language	Required by the course; large ecosystem
.venv	Virtual environment	Isolates packages from other projects
Ollama	Local model runtime	Easy Windows install, no API key
llama3.2:1b	LLM (~1.3 GB)	Fits 8 GB RAM on CPU
FastAPI	REST backend	Automatic OpenAPI/Swagger, Pydantic validation
Uvicorn	ASGI server	Runs FastAPI on port 8000

Streamlit	Web UI	Fast chat interface without React
httpx / requests	HTTP clients	Backend → Ollama; frontend → backend
pytest	Testing	Ten automated API tests
Markdown FAQ	Knowledge base	Easy to edit without a database

We also added conversation history (last 3 turns), streaming answers, and Good/Average/Poor ratings saved to data/feedback.jsonl

We did not use vector databases or embedding models because they would add RAM use and setup time beyond what our laptop could handle comfortably.

4. System architecture

Everything runs on **one computer** in three parts:

Part	Port	Role
Streamlit (chat page)	8501	What the student sees
FastAPI (backend)	8000	Validates requests, runs RAG, calls Ollama
Ollama (model)	11434	Generates the answer text

The frontend never calls Ollama directly. All model access goes through FastAPI. This matches how many real products are structured: UI → API gateway → model service.

4.1 Request flow (step by step)

1. Student types a question in Streamlit and presses Enter.
2. Streamlit shows a spinner or stream area while waiting.
3. Streamlit sends JSON to POST /ask(or /ask/streamif streaming is on).
4. FastAPI validates the body with Pydantic (empty question → HTTP 422).
5. If RAG is enabled, rag.pysearches the FAQ for matching sections.
6. FastAPI builds a message list: system prompt + FAQ context + optional history + user question.
7. llm_client.pysends the request to Ollama POST /api/chat.
8. Ollama returns generated text (30–90 seconds on our CPU).
9. FastAPI logs the exchange to backend/logs/app.log.
10. FastAPI returns JSON to Streamlit; the student sees the answer and can rate it.

4.2 Main source files

File	Responsibility
backend/main.py	Routes, models, error responses
backend/llm_client.py	Ollama HTTP calls, streaming
backend/rag.py	Split FAQ by headings, keyword match
backend/config.py	Model name, prompts, paths, timeouts
frontend/app.py	Chat UI, sidebar settings, ratings
data/university_faq.md	Demo knowledge base

5. RAG and prompt engineering

5.1 FAQ knowledge base

We wrote data/university_faq.mdwith sections a UDSM student might ask about:

FAQ section	Topics covered
Course Registration	ARIS portal, registration window, slip
Examinations	Rules, special exams, misconduct
Library Services	Hours, OPAC, borrowing limits
ICT Support	Email, Wi-Fi, student portal
Hostel Accommodation	Application, allocation, rules
Fee Payment	GePG, control number, banks
Academic Calendar	Semester dates (demo)
Student Conduct	Regulations, Dean of Students

Each section uses ## headings. The RAG module splits the file at those headings and treats each block as one searchable chunk.

5.2 How keyword RAG works

When a question arrives, we:

1. Lowercase the question and split it into words (ignore very short words).
2. Score each FAQ section by counting how many question words appear in that section.
3. Take the top one or two sections and insert them into the prompt as **context**.
4. Tell the model to prefer the context and admit when the FAQ does not contain the answer.

This is simpler than embedding-based RAG but worked for our assignment and did not need extra libraries or GPU memory.

5.3 Original vs improved prompt

Original prompt: You are a helpful assistant. - too generic; the model may guess or talk about topics outside UDSM.

Improved prompt (IMPROVED_PROMPTin config.py):

- Names the assistant as UDSM Student Support.
- Lists allowed topics (ARIS, exams, library, ICT, hostel, fees, calendar, conduct).
- Requires use of FAQ context when provided.
- Requires the phrase *"I don't have that information in the UDSM FAQ..."* when context is missing.
- Tells the model not to invent deadlines, fees, or official policies.

Screenshot 12 shows the difference for the question *What are UDSM library hours?* — the improved prompt produces structured UDSM-specific text with services listed from our FAQ.

5.4 Model settings

In config.pywe set:

Setting	Value	Reason
MODEL_NAME	llama3.2:1b	Small enough for 8 GB RAM
num_ctx	1024	Limit context size for memory
num_predict	256	Cap answer length
temperature	0.3	More factual, less random
REQUEST_TIMEOUT	120 s	CPU inference can be slow
MAX_HISTORY_TURNS	3	Keep prompt size manageable

6. Implementation by assignment task

Environment (Task 1) - venv, requirements.txt, run_backend.bat, run_frontend.bat, run_tests.bat. Evidence: screenshots 01–02.

Local LLM (Task 2) - ollama pull llama3.2:1b, model config in config.py. Evidence: screenshots 03–04. Screenshot 04 shows raw Ollama in the terminal (without RAG); the small model can hallucinate, which is why we added FAQ + improved prompt in the app.

FastAPI backend (Task 3) - /health, /ask, /ask/stream, /feedback, /feedback/summary, /prompts. Evidence: screenshots 05–08.

API summary:

Method	Path	Purpose
GET	/health	Check backend and Ollama status
POST	/ask	Full answer in one JSON response
POST	/ask/stream	Server-sent events, token by token
POST	/feedback	Save Good / Average / Poor rating
GET	/feedback/summary	Count ratings
GET	/prompts	Show original vs improved system prompt

Frontend (Task 4) - Streamlit chat, sidebar (RAG toggle, history, streaming, prompt version), help page. Evidence: screenshots 09–10.

Tests (Task 5) - pytest tests/ -v gives 10 passed when Ollama is running. Evidence: screenshot 11.

Improved prompt (Task 6) - IMPROVED_PROMPT in config.py. Evidence: screenshot 12.

Error handling (Task 7) - connection errors in Streamlit; HTTP 503 if Ollama is down; HTTP 422 for empty questions; spinner while waiting. Evidence: screenshot 13.

Logging (Task 8) - backend/logs/app.log records questions, RAG flags, Ollama HTTP calls, and answer previews. Evidence: screenshot 14.

Reflection (Task 9) - see Section 12 and docs/submit_reflection.pdf.

Bonus features - RAG, ratings, history, streaming. Evidence: screenshots 15–16.

7. Backend and frontend details

7.1 Backend behaviour

"/health" returns status, model, ollama_reachable, and a short message. If Ollama is stopped, status becomes degraded so we can detect problems before a long /ask wait.

"/ask" accepts:

```
{
  "question": "string (required)",
  "use_rag": true, "prompt_version":
  "improved",
  "history": [{"role": "user", "content": "..."}, {"role": "assistant", "con-
tent": "..."}]
}
```

The response includes answer, used_rag, context_used, prompt_version, history_used, and timestamp.

"/ask/stream" returns Server-Sent Events (data: {"token": "..."}) until data: [DONE]. The Streamlit UI uses st.write_stream to show tokens as they arrive.

"/feedback" appends one JSON line per rating to data/feedback.jsonl with question, answer snippet, rating, and timestamp.

7.2 Frontend behaviour

The Streamlit app has two pages: **Chat** and **Help & Documentation**.

On the chat page:

- **Backend URL** — default http://127.0.0.1:8000; can be changed in the sidebar.
- **Use FAQ (RAG)** — when checked, backend retrieves FAQ context.
- **Use conversation history** — sends last turns to the model for follow-up questions.
- **Stream response** — uses /ask/streaminstead of waiting for one JSON blob.
- **Prompt version** — switch between originaland improved.
- **Rate this answer** — Good / Average / Poor buttons after each reply.

If the backend is unreachable, the UI shows: *"Cannot connect to backend. Is FastAPI running on port 8000?"* (screenshot 13).

8. Installation and how to run

These are the steps we used on Windows (also documented in README.md):

1. Clone or unzip the project folder.
2. Create and activate venv: python -m venv .venvthen .\.venv\Scripts\Activate.ps1
3. Install packages: pip install -r requirements.txt
4. Install Ollama from https://ollama.com and run: ollama pull llama3.2:1b
5. **Terminal 1 — backend:** uvicorn backend.main:app --host 127.0.0.1 --port 8000
6. **Terminal 2 — frontend:** streamlit run frontend/app.py
7. Open browser at http://localhost:8501and ask a test question.

Quick test without UI:

- Browser: http://127.0.0.1:8000/health

- Swagger: http://127.0.0.1:8000/docs
- Tests: pytest tests/ -v(Ollama must be running for the live /asktests)

Batch files run_backend.bat, run_frontend.bat, and run_tests.bat wrap the same commands for convenience.

9. Testing and results

9.1 Automated tests

All ten pytest tests pass with Ollama running:

Test	What it checks
test_health_returns_ok	/healthreturns 200
test_ask_empty_question_returns_422	Validation on empty input
test_prompts_endpoint	/promptsreturns both prompt versions
test_feedback_saved	POST /feedbackwrites to file
test_feedback_summary	GET /feedback/summaryreturns counts
test_ask_invalid_history_role_returns_422	Bad history role rejected
test_trim_history_caps_turns	History trimmed to 3 turns
test_ask_valid_question	Live /askwith Ollama
test_ask_with_history	Multi-turn request works
test_stream_endpoint	/ask/streamreturns SSE tokens

Total time on our laptop: about **83 seconds** (screenshot 11), mostly because live Ollama calls are slow on CPU.

9.2 Manual test cases

Scenario	Steps	Expected result	Evidence
Happy path	Ask registration question in UI	Answer about ARIS	Screenshot 10
Health check	Open /healthin browser	ollama_reachable: true	Screenshot 07
API via Swagger	POST /askin /docs	200 + JSON answer	Screenshot 08
Backend down	Wrong backend URL in sidebar	Red connection error	Screenshot 13

Streaming	Enable stream, ask library hours	Answer appears incrementally	Screenshot 15
History	Two related questions via API	Second answer uses context	Screenshot 16
Logging	Ask any question	New lines in app.log	Screenshot 14

9.3 Screenshot verification (01–16)

#	File	Verified content
01	01-venv-activated.png	.venvactive, Python version shown
02	02-pip-install.png	pip install -r require-ments.txt, exit code 0
03	03-ollama-pull.png	llama3.2:1blisted in ollama list
04	04-ollama-run.png	Model responds in terminal (raw Ollama)
05	05-fastapi-running.png	Uvicorn on http://127.0.0.1:8000
06	06-swagger-docs.png	UDSM API at /docs
07	07-health-response.png	JSON: status: ok, olla-ma_reachable: true
08	08-ask-response.png	POST /ask200 with regis-tration answer
09	09-frontend-home.png	Streamlit chat, sidebar set-tings
10	10-frontend-qa.png	GePG fees question with answer and ratings
11	11-pytest-results.png	**10 passed**
12	12-prompt-comparison.png	Generic vs improved prompt
13	13-error-handling.png	Connection error when backend URL wrong
14	14-log-file.png	app.logwith question + Ollama lines
15	15-streaming-demo.png	Streamed library-hours an-swer
16	16-chat-history.png	Two-turn Q&A, histo ry_used: true

All PNG files are in docs/screenshots/.

10. Error handling and logging

10.1 Error handling

Situation	System response
Empty question	HTTP 422 from API; warning in Streamlit
Ollama not running	HTTP 503 from /ask; health shows degrad-ed
Backend not running	Streamlit connection error message
Invalid history role	HTTP 422
Request timeout (120 s)	Error shown to user after timeout

We chose clear messages instead of silent failures so a student or examiner can see what went wrong during a demo.

10.2 Logging

backend/logs/app.log uses Python logging with timestamps. Typical lines:

- Received question: ... | RAG=True | prompt=improved
- HTTP Request: POST http://localhost:11434/api/chat
- Generated answer for '...': (first part of answer)

Logs helped us debug slow responses and confirm RAG was on during tests. In production, log access would need privacy rules because questions may contain personal details.

10.3 Feedback file

Ratings are stored in data/feedback.jsonl (one JSON object per line). The sidebar can show a simple summary count. This is a basic quality loop - staff could later read Poor ratings to improve the FAQ.

11. Challenges and how we addressed them

RAM (8 GB): Running Ollama, FastAPI, Streamlit, and a browser together was tight. We used llama3.2:1b, limited context (num_ctx: 1024), and closed other apps during demos.

CPU speed: Answers often took 30-90 seconds. We added user-facing wait messages, a 120-second timeout, and optional streaming so the user sees progress.

Ollama install size: First download needed stable internet; after that the model runs offline.

FAQ writing: No official dataset was provided, so we wrote realistic demo content and state clearly it is not policy.

Windows/PowerShell: We used ;instead of &&and .batscripts to reduce setup mistakes.

Streaming bug: Early version passed a lambda incorrectly to st.write_stream; we fixed it to pass the generator directly and recaptured screenshot 15.

12. Production readiness (Task 9 summary)

This project is a **prototype on one laptop**, not something UDSM could publish tomorrow.

If the university wanted a real version, we would need at least:

- HTTPS and student login (example. linked to ARIS)
- Official verified documents instead of our demo FAQ
- A server or GPU for faster answers and more concurrent users
- Monitoring (/healthchecks, error alerts, latency metrics)
- Rate limiting and abuse protection
- Data retention policy for logs and feedback
- Human escalation when the bot is unsure

Local vs cloud API: Local keeps data on the machine and avoids per-token cost, but is slower and uses weaker models. Cloud APIs are faster and stronger but cost money and send data off-device.

Security today: No authentication, no HTTPS on localhost, plain-text logs, and the model can still hallucinate even with RAG.

Full answers to all ten reflection questions are in **docs/submit_reflection.pdf`**.

13. What we learned

Working on this project, our group learned practical lessons that match IS 365 outcomes:

1. **An LLM app is a system**, not a single model call - we needed API design, UI, config, and tests.
2. **Small models need help** - FAQ retrieval and a strict prompt improved answers more than switching to a bigger model we could not run.
3. **Validation matters** - Pydantic and pytest caught empty input and bad history before demo day.
4. **Observability matters** - logs and /health saved time when Ollama or the backend was not started.
5. **Deployment is harder than a demo** - authentication, HTTPS, and official content would be the next step for UDSM.

These points are developed further in our reflection document.

14. Conclusion

We completed a working UDSM Student Support Assistant using a local LLM pipeline: Ollama for inference, FastAPI for the API layer, Streamlit for the chat UI, keyword RAG on a UDSM FAQ file, improved prompting, automated tests, error handling, and logging.

The system meets the practical requirements and is suitable for demonstration. Significant work would still be needed before real student use at UDSM — especially verified content, security, and infrastructure.

This repository includes this report, the reflection document, source code, and the docs/ folder with screenshots 01–16 and supporting markdown files.

15. Appendix

A. Dependencies ("requirements.txt")

Package	Use
fastapi	REST API framework
uvicorn	ASGI server
httpx	Async HTTP (tests, Ollama client)
pydantic-settings	Configuration
streamlit	Chat frontend
requests	Frontend HTTP calls
pytest	Automated tests

B. Sample code - health check

```
@app.get("/health") def
health_check():
    ollama_ok = check_ollama() return
    {
        "status": "ok" if ollama_ok else "degraded", "model":
        MODEL_NAME,
        "ollama_reachable": ollama_ok,
    }
```

C. Sample code - RAG scoring (simplified)

```
def retrieve_context(question: str, top_k: int = 2) -> str: words =
    [w.lower() for w in question.split() if len(w) > 2] scored = []
    for section in faq_sections:
        score = sum(1 for w in words if w in section.lower()) scored.append((score,
        section))
    scored.sort(reverse=True)
    return "\n\n".join(s for s, _ in scored[:top_k] if _ > 0)
```

D. Improved prompt (excerpt)

The full text is in backend/config.py. It defines the assistant role, allowed UDSM topics, FAQ usage rules, and a standard phrase when information is missing.

E. Screenshot index

All evidence PNGs: docs/screenshots/01-venv-activated.png through 16-chat-history.png. See Section 9.3 for the verification table.

F. Related documents (“docs/”)

Document	**Purpose**
prompt_comparison.md	Task 6 - prompt comparison
error_handling.md	Task 7 - error handling
testing.md	API and pytest evidence
architecture.md	System architecture
bonuses.md	Bonus features
learning_outcomes.md	Course outcome mapping
submit_reflection.md	Task 9 reflection
screenshots/README.md	Evidence image index